

R-BurstHADS: Record of Changes, Corrections and Additions

A plain language account of the work carried out on the simulator, the experimental design and the paper.

1. Why this work was carried out

Before writing the paper we audited the whole project from the two source papers down to our own code and our own experimental method. The purpose was to make sure that every number we intended to publish could survive a reviewer checking it. The audit compared our implementation against the published descriptions of HADS and Burst HADS, checked the simulator for internal consistency, and tested whether our experiment was actually capable of telling the three schedulers apart. It found problems in all three areas, and this document records what they were and what we did about them.

2. What we confirmed was already correct

Several things were checked carefully and found to need no change. Recording these matters because it shows the audit was even handed rather than a search for faults.

The workload model uses independent tasks with no dependencies between them. We confirmed from both source papers that they study Bag of Tasks applications, which are independent by definition, so our model is faithful and we did not change it.

The cost metric already charged each virtual machine for every second it was running, which matches the billing model described in the Burst HADS paper. The task execution times and memory sizes we generate also match the ranges published for the reference jobs.

We verified the rule that a burstable machine handles only one task at a time. The source paper does not state this as a sentence, so we read the pseudocode of its migration procedure and confirmed it structurally. A burstable machine is only ever chosen from the idle set, and the moment it receives a task it moves to the busy set, so it can never take a second one. Our implementation already behaved this way in both operating modes and was left unchanged.

3. Fairness problems in the comparison

These were the most serious findings, because they meant our proposed method was being compared against baselines that were handicapped.

3.1 Our method had access to a machine type the baselines did not

R BurstHADS created c5.xlarge machines when it needed extra capacity, but that machine type was not present in the shared pool the three schedulers drew from. Neither baseline had any way to select it.

Because c5.xlarge is the most cost efficient machine in the study, this alone accounted for essentially the entire advantage we were reporting. When we forced our method to use the same machine the baselines use, it became bit for bit identical to Burst HADS. We traced the cause: an older version of the pool did contain that machine type, the pool was later rewritten without it, and the hard coded constants inside our scheduler were never updated to match. We corrected this by restoring the machine type to the shared pool and by making our scheduler choose what to provision from whatever pool it is given, so it can never obtain something the baselines could not.

3.2 The baseline was given a feature belonging to the method under test

Work stealing is a contribution of the Burst HADS paper, not of HADS. Our HADS implementation had it switched on, which erased a large part of the advantage Burst HADS is supposed to show. We switched it off in HADS, where it does not belong.

3.3 Our replacement machines could never be interrupted

The simulator decided which machines would hibernate at the very start of the run. Any machine created later, which means every replacement our method provisions, therefore faced no possibility of interruption at all. Our method was buying spot machines at spot prices that behaved like guaranteed machines. We fixed this so that every spot machine created during a run is entered into the same random interruption process as the machines that were there from the beginning.

4. Correctness problems in the simulator

4.1 The deadline was far too loose for the experiment to mean anything

The deadline was calculated by dividing the total work by a count of machines, which ignored both how fast each machine was and how many tasks it could run at once. The pool actually retires thirty six units of work per second, not three, so the deadline came out roughly twelve times larger than the fastest possible completion time. The consequence was severe. Across all two thousand seven hundred runs of the earlier sweep, not one scheduler ever missed a deadline. Worse, both baselines only buy extra capacity when the deadline is threatened, so a deadline that could never be threatened meant the baselines never reacted at all, and the gap we were measuring was largely an artefact of that. We rewrote the calculation to derive the deadline from the pool's real throughput, and calibrated it by measurement so that a well organised undisturbed run meets it but a disrupted one does not without buying capacity.

4.2 A related calculation made one scenario do nothing at all

The estimate of expected run length was too large by a factor of between one and a half and three. The late hibernation scenario places its interruption at forty percent of that estimate, so at the largest workload the interruption was scheduled for a moment after the run had already finished. That scenario was silently doing nothing. Correcting the estimate fixed it.

4.3 The instance ranking formula used the wrong quantity

The scheduling formula that ranks candidate machines is defined in the paper as the computing power of the whole machine divided by its price. Our code used the speed of a single core instead of the whole

machine, which quietly discarded the number of cores. The effect was not cosmetic. It rated a two core machine and a four core machine as exactly equal when the second does twice the work per rupee, and because ties are broken by taking the first candidate, the scheduler opened the slowest and least efficient machine in the pool first and then packed everything into it. This is what made HADS look pathologically bad. Correcting the formula to use whole machine capacity restored sensible behaviour and made HADS a credible baseline.

4.4 Interruption was modelled as a benefit rather than a cost

Tasks pay a ten percent overhead for checkpointing. The condition controlling this was reversed, so the overhead was charged on a task's first undisturbed run and skipped once it had been interrupted. Recovering from a hibernation was therefore cheaper per unit of work than never being interrupted at all. Combined with billing correctly stopping during hibernation, this made being interrupted a net advantage, which is why HADS appeared to finish faster when we broke it. We made the checkpointing overhead a standing cost that applies whenever a task runs.

4.5 The planner and the executor disagreed with each other

After the previous correction, the function that predicts when a task will finish no longer charged the overhead that actual execution charges. Every placement decision was therefore ten percent optimistic, and whichever scheduler packed closest to the deadline overshot it. Burst HADS stopped opening new machines at three, packed around ninety tasks onto each, and missed eight deadlines, while HADS opened a fourth machine and missed none. This was purely an artefact of the prediction. Once the two were brought into agreement, Burst HADS opened four machines, missed nothing, and became faster than HADS as the source paper says it should be.

4.6 The search removed its own safety constraint

The local search is allowed to relax its internal deadline when it stops improving. The paper applies this once per stalled window, but our loop applied it on every iteration while stalled, so the value compounded. We measured it growing from one thousand six hundred and eighty four seconds to three point four times ten to the power eighteen. At that point the constraint no longer existed. This matters because that constraint is precisely the reserve that guarantees enough time to move tasks off a hibernated machine. We bounded the relaxation to once per stalled window and capped it at the real deadline.

4.7 Tasks were being silently discarded

Our Burst HADS placement routine tried spot machines and then simply gave up. Any task that fitted nowhere was left out of the schedule entirely, never assigned and never run. Because our measurements skip tasks that never finished, such a task contributed no cost, no time and no deadline miss. It vanished from the results instead of appearing as a failure. This showed up as a completion rate of ninety two percent, with the cost and makespan for those cells calculated over the surviving ninety two percent only. We added the missing final step, which falls back to on demand machines and raises an error if even a brand new machine cannot finish the task in time. Completion is now one hundred percent everywhere.

4.8 A rescue step was making tasks worse instead of better

Burst HADS moves tasks predicted to run late onto burstable machines. Our code checked only whether the machine had enough memory, never whether the move actually helped. Burstable machines in their economy mode run at a fifth of normal speed, so a task that was slightly late could be relocated and become enormously late. We measured a task of three hundred and forty eight seconds being moved and then taking nine hundred and fifty seven seconds, single handedly setting the makespan and missing a deadline every other task met. We added the obvious condition that the move must be an improvement.

5. Problems with the experimental design

5.1 Our hibernation scenarios were not the ones in the papers

We had invented our own scenarios, the most important of which took every spot machine down simultaneously. That scenario appears in neither source paper, and it is outside the range where the burstable rescue mechanism can do anything at all, because with the entire spot pool gone both schedulers fall through to on demand machines. We replaced our scenarios with the five defined in the Burst HADS paper, which model interruptions as independent random events per machine with published rates, and which include recovery in three of the five cases. Our earlier code never modelled recovery at all, so we could only ever express two of the five.

5.2 The interruption rule punished good behaviour

Our rule chose to hibernate whichever machine a scheduler had loaded most heavily. This sounds reasonable but is an adversarial rule, because it rewards leaving your fastest machine empty. HADS packed everything onto one machine and left another untouched, and the rule then spared exactly that untouched machine, which went on to absorb most of the rescued work. Burst HADS spread its load, so the rule destroyed both of its strong machines. A scheduler was being punished for balancing its load. Real interruption depends on the machine type and the market, never on how busy you happen to have made it, so we replaced this with a rule that does not look at load.

5.3 The pool was too small for the mechanism to show itself

HADS packs onto as few machines as the deadline forces. Burst HADS uses a search that balances cost against time, so it spreads out to buy speed. That difference is the entire reason Burst HADS is faster, and it can only appear if there are spare machines left to spread onto. With one machine of each type both schedulers were capacity bound and the published gap collapsed to nothing. We enlarged the pool to three of each spot type, and the gap reappeared. The deadline scales automatically with the pool, so enlarging it does not make the problem easier.

6. What we added

The deadline aware provisioning rule is the substantive addition to the method itself. Previously our scheduler bought capacity according to a fixed constant, eight tasks per machine, which was fitted to this workload and took no account of how much time remained. We measured the consequence: twenty

one machines provisioned, seventy six percent of their paid time sitting idle, and killing seventeen of them changed the finishing time by zero point zero percent while cutting cost by forty point seven percent. They were contributing nothing. We replaced the constant with a calculation based on conservation of work, which asks how much work remains, how much time is left, and how much throughput is already available, and then buys only the shortfall. It now buys one machine where it used to buy twenty one, and idle waste fell from seventy six percent to eight percent.

We also added a validation harness that reproduces the published comparison between HADS and Burst HADS using their own machine catalogue, jobs, deadline and scenarios. This is what allows the paper to argue that our baselines are trustworthy before it claims anything against them.

Finally we wrote a new figure generator that reads the results file directly, so figures can be regenerated in about one second without running any simulation. It produces ten figures covering makespan, cost, cost per task, speedup, deadline utilisation, variability across seeds, the cost and time trade off, a per cell breakdown, sensitivity to the interruption rate, and the validation comparison. The two older figure scripts should not be used, because they compare only two schedulers, use the retired scenarios, and do not reset the random number generator between schedulers, which we measured causing swings of fifteen to twenty two percent depending purely on the order in which the schedulers happened to run.

7. What the corrected experiment now shows

All three schedulers complete every task in every configuration. Burst HADS and R BurstHADS meet the deadline in all thirty configurations, while HADS misses in three of them, at the workload size where the deadline is tightest and HADS has no burstable rescue capacity to fall back on.

At the largest workload the ordering reported in the source paper is reproduced in all five scenarios. Burst HADS is faster than HADS and slightly more expensive, which is what that paper itself reports, and not, as we had assumed earlier, cheaper.

Our own method is between twenty six and forty percent faster than Burst HADS in every scenario, and its cost advantage grows steadily with the interruption rate. It is cost neutral when interruptions are rare, twelve point five percent cheaper at the middle rate, and between fourteen and sixty four percent cheaper at the highest rate. The fact that this advantage tracks a parameter we never tuned, and which describes the environment rather than our algorithm, is the strongest evidence that the effect is real rather than a product of how we configured things.

8. What we are stating honestly as limitations

Our reproduction of the published makespan improvement is smaller than the published figure, and the shortfall grows with job size. The likely reason is that at a fixed deadline the largest job consumes the whole pool, leaving the search nowhere to spread. We report this as a limitation of our reproduction rather than as a correction to the original authors.

The task lengths are drawn from a uniform distribution whose range matches the reference jobs but whose shape does not reflect the long tail of real workloads. Hibernation stops compute billing but we

do not charge for the retained storage volume, which the provider does charge for, although the amount is small. The different interruption rates we assign to different machine types are a modelling choice of ours and not a measured quantity, and the paper says so plainly.